ChatGPT

사내 정보 검색 및 코드 자동화 최신 기술 연구

1. 사내 문서 검색을 위한 RAG 기반 챗봇 기술

PDF 기반 RAG(ChatGPT 등의 Retrieval-Augmented Generation) 챗봇의 필요성: 대형 언어모델(LLM)을 사내 지식 검색에 활용할 때 RAG 기법이 필수적이다. RAG는 모델 답변에 조직의 **자체 문서를 인용하여 정보의 사실성을 높이고 환각(hallucination)을 줄여준다** [1]. 예컨대, 기업들은 높은 정확도와 투명성이 요구되는 내부 Q&A에 RAG를 적용하며, **모델 출력의 근거를 내부 데이터로 제시함으로써 신뢰성과 보안성을 확보**하고 있다 [1]. Notion의 공동창업자 Ivan Zhao는 2024년 인터뷰에서 **"RAG는 지식 관리 방식을 완전히 바꾸어 팀을 정보 정리의 수고에서 해방시키고, 개인이나 회사에 완벽한 기억력을 제공한다"고 언급**했다 [2]. 과거에는 키워드로 정확히 검색해야 정보를 찾았지만, **이제 LLM이 자연어 질문을 이해하고 관련 정보를 찾아주므로 사용자는 일일이 문서를 분류하거나 동료에게 물어볼 필요가 없다** [3] [4]. 실제로 Notion 사내 도입 결과, 팀원이 "우리 회사 이사가 언제였지?" 같은 질문을 하면, 기록된 문서를 찾아 몇 초 만에 답변해주어 **기존에 답을 찾고 이메일로 문의하던 수십 분의 작업을 단축**했다고 한다 [4]. 이처럼 **RAG 기반 챗봇은 방대한 사내 PDF와 지식의 "완벽한 기억" 역할**을 하여 업무 생산성을 높인다 [5] [6].

최근 기업들의 적용 사례: 2023년 이후 여러 기업이 자사 플랫폼에 RAG 챗봇을 도입하였다. **Notion**은 2023년 말 Notion AI Q&A 기능을 출시하여, 사용자가 Notion 워크스페이스 내 문서와 메모에 자연어로 질문하면 바로 답을 얻을 수 있게 했다. 2024년 업데이트에서는 **Notion AI가 Slack, Google Drive, Jira, Salesforce 등 외부 도구까지 검색**하여 "팀의 모든 지식을 빠르게 조회"하는 기능이 추가되었다 [7]. Notion은 이 기능을 "회사의 기억을 해제(Unlock your company's memory)"한다고 표현하며, **문서와 메시지 곳곳에 흩어진 암묵지를 AI가 끌어모아 팀의 의사결정을 돕는다**고 밝혔다 [8]. **Slack GPT**도 2023년 Salesforce에 의해 발표된 사례다. Slack GPT는 Slack 내 대화 요약이나 메시지 작성 보조뿐 아니라, **기업의 신뢰할 수 있는 내부 지식과 고객 데이터를 연계한 대화형 AI 플랫폼**을 표방했다 [9]. Slack CEO 리디아네 존스는 "AI의 진정한 힘은 기업의 내부 지식과 신뢰받는 데이터를 분석·활용하는 데 있다. Slack GPT는 조직이 자체 데이터와 직원 지식을 손쉽게 활용하도록 도와 더 빠른 의사결정을 가능케 한다"고 강조했다 [9]. 한편 **SAP**는 자사 업무 소프트웨어에 AI 조력자 SAP Joule을 통합하고 있다 (2023년 발표). Joule은 **SAP 내부 데이터뿐 아니라 SharePoint 등의 외부 문서까지 검색하는 "문서 그라운딩(document grounding)" 기능**을 갖춰, 예를 들어 **인사부서 직원이 인사 정책 PDF를 질문하면 바로 관련 조항을 찾아 답변**해준다 [10]. SAP는 이러한 **생성형 AI 기반 Q&A가 직원들이 원하는 정보를 즉시 얻음으로써 업무 효율과 경험을 크게 높인다**고 보고하였다 [10]. 이처럼 Notion, Slack, SAP 사례에서 보듯, **현대 기업들은 사내 지식 관리에 RAG형 챗봇을 도입해** 직원들이 방대한 내부 정보를 자연어로 질의응답하고, 문서 요약을 자동화하며, 궁극적으로 **"일하는 방식" 자체를 혁신**하고 있다 [4] [9].

오픈소스 vs SaaS 솔루션 비교: 현재 **RAG 구현을 위한 다양한 프레임워크**와 솔루션들이 등장하고 있다. 오픈소스 생태계에서는 **LangChain, LlamaIndex, Haystack** 등이 대표적이다. LangChain (2023)은 파이썬 기반 LLM 오케스트레이션 프레임워크로, **다양한 도구·API·모델을 체인으로 묶어 복잡한 작업을 구성**할 수 있다 [11] [12]. 유연성과 통합력이 뛰어나 많은 스타트업들이 사용하지만, **비개발자에게는 다소 학습곡선이 가파르고** 복잡한 체인을 설계하는 데 기술 전문성이 필요하다 [13]. LlamaIndex (구 GPT Index, 2023) 역시 주목받는 오픈소스로, **PDF, 웹, DB 등 복잡한 데이터 구조를 인덱싱하여 LLM 질의에 활용**하도록 돕는다 [14] [15]. LlamaIndex는 "똑똑한 사서"처럼 데이터를 조직화하고, 자연어로 데이터베이스 질의도 가능하게 하여 **방대한 사내 문서에서 정확한 정보를 신속히 찾아주는** 데 특화되어 있다 [16] [17]. Haystack (deepset사의 프레임워크)은 **엔터프라이즈급 QA 시스템에 적합한 종단간 솔루션**으로, 검색 파이프라인(문서 리트리버+리더+랭커)을 제공하고 대용량 문서에서도 실시간 질의응답이 가능하다 [18] [19]. Haystack은 Elasticsearch 등 전통 검색엔진이나 OpenAI API와도 연동되며, **멀티언어 및 의미기반 검색을 지원**하여 글로벌 기업 활용에 강점이 있다 [20] [21]. 이러한 오픈소스 도구들은 **자체 서버에 설치하여 데이터가 외부로 나가지 않게 통제**할 수 있고, 세밀한 커스터마이징이 가능하다는 장점이 있다. 반면 초기 구축에 여러 컴포넌트 통합과 벡터DB 세팅 등이 필요해 **도입 난이도가 있다**. Vectara의 2024년 보고에 따르면, **직접 RAG 시스템을 구축하면 20개 이상의 API와 5~10개의 벤더를 관리해야 할 정도로 복잡**하며, 이에 따라 이를 단순화한 상용 플랫폼도 늘고 있다고 한다 [22].

한편 **클라우드/SaaS로는 벡터DB 및 완성형 API 서비스**들이 각광받고 있다. 예를 들어 Pinecone, Weaviate 같은 벡터 데이터베이스 서비스를 활용하면 손쉽게 임베딩 기반 문서 검색을 구현할 수 있고, OpenAI의 ChatGPT Retrieval Plugin이나 Azure의 Cognitive Search는 플러그인 형태로 사내 데이터 질의응답을 제공한다. 이러한 SaaS 솔루션은 **구축의 편의성**이 높고 관리 부담을 줄여주지만, 민감한 내부 데이터를 외부 서비스에 보내야 하는 **프라이버시 이슈**를 신중히 고려해야 한다. 다행히 OpenAI 등의 업체는 "API로 제출된 기업 데이터는 모델 학습에 사용되지 않는다"고 명시하고 데이터 암호화·보존기간 제한 등의 **기업용 프라이버시 보호조치**를 제공하기 시작했다 [23]. 그럼에도 금융/의료같이 극도로 민감한 데이터는 아예 내재화된 오픈소스 솔루션으로 처리하거나, 최소한 **프라이빗 클라우드에 호스팅되는 SaaS**를 선호하는 추세다.

성능 및 도입 고려사항: 2023~2024년 사이 RAG 성능은 **LLM 모델 향상과 함께 크게 진보**했다. 예를 들어 OpenAI GPT-4 및 Anthropic Claude 2 등은 이전 세대보다 컨텍스트 창이 대폭 증가하여 (Claude 100k는 무려 10만 토큰 입력 지원) 수백 페이지 분량의 PDF도 모델에 통째로 제공해 요약·분석하는 것이 가능해졌다 [24] [25]. 이러한 초대용량 컨텍스트 LLM은 복잡한 질의에 대해 **벡터 검색 없이도 모델 자체 메모리로 답변하는 실험**도 등장했지만 [25], 현실적으로 **다수 문서에 걸친 질의나 최신 데이터 활용에는 여전히 RAG가 효과적**이다. RAG는 필요한 부분만 검색해 **프롬프트 크기를 줄이고 응답 정확도를 높이는 장점**이 있기 때문이다 [26]. 실제 기업 도입 시에는 **응답 지연**과 **비용**도 고려해야 하는데, 2024년 기준 LLM 추론 속도는 전년 대비 7배 빨라져 상용 앱에서도 실사용 가능한 수준이 되었고 [27], 벡터DB 등 인프라도 최적화가 진행되어 왔다. 마지막으로 **보안/프라이버시** 측면에서, RAG 시스템 구축시 **사내 권한체계와 데이터 암호화**를 반드시 통합해야 한다. 이는 아래 3번 항목에서 다루겠지만, **RAG 도구 자체에 접근통제 기능이 부족한 경우 별도 계층의 필터링을 구현**해서라도 사용자별로 검색 가능한 문서를 제한해야 한다 [28]. 종합하면, RAG 기반 사내 챗봇 기술은 **다양한 도구들의 성숙과 함께 현실적인 도입 사례**가 늘고 있으며, 성능 또한 개선되었다. 기업은 오픈소스와 SaaS 옵션 중 **자사 보안 요건과 개발 역량에 맞게 선택**하고, 도입 시 **모델 성능뿐 아니라 데이터 유출 위험 방지와 사용자 권한 설정**까지 총체적으로 검토하는 것이 중요하다.

2. GitHub 코드 리포지토리 연동 코드 설명 생성 기술

함수 주석/설명 자동 생성의 현황: 최근 1년간 **코드 전용 LLM**들이 등장하며, Python, Java, C 등 주요 언어의 함수에 대한 **자동 주석 생성**이 현실화되고 있다. 과거에도 코드 요약 연구가 있었지만, 2023년의 최신 모델들은 수십 억~수백억 파라미터 규모로 훈련되어 **사람 수준에 가까운 코드 이해와 자연어 설명**을 보여준다. 예를 들어, Meta가 2023년 8월 공개한 **Code Llama**는 Llama2를 프로그래밍 데이터로 정밀 튜닝한 모델로서, **주석이 부족한 코드를 이해하고 자연어로 설명하거나 보완하는 데 뛰어난 성능**을 보였다 [29]. Meta는 Code Llama를 발표하며 "공개된 LLM 중 코딩 작업에서 최첨단 수준(state-of-the-art)"이며 개발자들이 **보다 견고하고 잘 문서화된 소프트웨어를 작성**하도록 도와줄 것이라 밝혔다 [29]. 이는 AI가 함수의 의도를 설명하고 주석을 생성함으로써 **코드 유지보수성과 가독성을 높이는 도구**가 될 수 있음을 시사한다.

최근 1년간 발표된 주요 모델 및 연구: 2023년에는 여러 기관에서 코드 특화 LLM 연구성과를 발표하였다. **Salesforce 연구팀**은 2023년 5월 CodeT5+라는 오픈소스 코드 LLM 패밀리를 공개했다 (Wang 등, 2023). CodeT5+는 2억~160억 규모의 인코더-디코더 모델로, span fill, contrastive learning 등 다양한 사전학습 목표를 도입하여 **코드 생성, 보완, 수식 문제 해결 등 20여개 코드 과제에서 새로운 SOTA 성능**을 달성했다 [30]. 특히 **CodeT5+ 16B 모델은 코드 생성 벤치마크 HumanEval에서 당시 공개된 다른 어떤 코드 LLM보다 높은 성공률(SOTA)을 기록**하였다 [30]. 또 다른 주목할 성과는 **BigCode 프로젝트**(ServiceNow-HuggingFace)의 StarCoder (Li 등, 2023)다. StarCoder는 **80여 종의 언어로 구성된 GitHub 데이터 1조 토큰**으로 학습된 155억 모델로, 다국어에 능하며 최대 8천 토큰 컨텍스트를 지원한다 [31]. 2023년 5월 발표된 논문에서 StarCoder는 "여러 프로그래밍 언어를 지원하는 모든 공개 코드 LLM 중 최고 성능을 보였고, OpenAI의 과거 코드 모델인 Codex (code-cushman-001) 수준을 대등하거나 능가"한다고 보고되었다 [32]. 또한 **Meta의 Code Llama**(7B/13B/34B)도 공개 후 오픈소스 커뮤니티에서 많이 활용되었고, 이를 기반으로 한 파생모델들이 등장했다. 그 중 **Phind**라는 스타트업은 2023년 10월 자사 모델 (Phind Code 34B)을 공개하며 "오픈소스 코드 LLM으로는 처음 GPT-4 수준의 코딩 능력을 넘어섰다"고 발표했다 [33]. Phind 모델은 Meta의 CodeLlama-34B를 파인튜닝한 것으로, HumanEval 테스트에서 GPT-4의 점수를 능가하는 **정확도 74.7%**를 달성했고 응답속도도 GPT-4보다 5배 빨랐다 (NVIDIA H100 기반) [33] [34]. 이처럼 **오픈소스 코드 모델의 약진**으로 개발자는 무료로 강력한 코드 생성/요약 AI를 활용할 수 있는 길이 열렸다. 한편, OpenAI의 GPT-4 같은 폐쇄형 모델도 뛰어난 코

드 이해력을 인정받아 GitHub Copilot 등에 채택되고 있으며, **Anthropic Claude 2 등도 최대 10만 토큰 맥락으로 대규모 코드베이스를 한꺼번에 읽어주는** 등 혁신을 보이고 있다.

연구 측면에서는, **코드 생성 + 실행 결합**에 대한 시도가 눈에 띈다. 2024년 3월 공개된 OpenCodeInterpreter 프로젝트(Xiang Yue 등, 2024)는 오픈모델에 **코드 실행 및 수정 피드백 루프**를 통합하여, OpenAI의 "코드 인터프리터"와 유사한 능력을 구현했다 [35]. OpenCodeInterpreter의 33억~330억 모델들은 HumanEval, MBPP 등 평가에서 **GPT-4 Code Interpreter에 맞먹는 83~84% 수준 정확도**를 달성했고, GPT-4로 생성한 피드백을 활용하면 90%대까지 성능이 향상되었다고 보고되었다 [35]. 이는 **모델이 코드 결과를 직접 실행하고 오류를 수정**하며 해설까지 이어가는 자동 프로그래밍을 실험한 것으로, 향후 코드 설명 생성 기술이 단순 주석을 넘어 **실행 검증된 예제 코드와 인터랙티브한 문서 생성**으로 발전할 가능성을 보여준다.

요약하면, **최근 1년간 CodeT5+, StarCoder, Code Llama, Phind 등 다수의 오픈소스 LLM과 연구 성과**가 코드 설명 생성의 정확도와 실용성을 크게 높였다. 이러한 모델들은 Python, Java, C, JavaScript 등 다양한 언어에 능통하도록 훈련되어, 함수 단위부터 대규모 프로젝트까지 **자동으로 맥락을 파악하고 적절한 주석, 문서, 요약을 생성**할 수 있다.

개발자 워크플로우에서의 활용: 코드 리딩 및 문서화 자동화는 이미 많은 개발자 도구에 통합되고 있다. 가장 대표적인 것은 **IDE 기반 AI 페어 프로그래머**이다. GitHub Copilot은 2021년부터 VS Code 등의 편집기에서 코드 자동완성을 제공해왔고, 2023년 Copilot X 업그레이드를 통해 **대화형 챗봇으로 코드 설명을 질문하거나, 작성한 함수에 대한 docstring을 생성**하는 기능을 도입했다. 예를 들어 개발자가 구현한 함수에 "이 함수의 기능을 설명하는 주석을 추가해줘"라고 프롬프트하면 Copilot이 함수 로직을 분석하여 한글 또는 영어로 요약 주석을 제안해준다. 또한 Copilot은 **Pull Request의 변경점을 요약하여 설명문을 써주는 기능**도 제공하기 시작했는데, 2024년 연구에 따르면 **AI가 생성한 PR 설명을 활용한 경우 코드 리뷰 시간이 단축되고 PR 병합 확률이 높아지는 효과**가 관측되었다 [36]. 실제 오픈소스 프로젝트 18,000여 건을 분석한 결과, Copilot PR 설명을 사용한 PR은 그렇지 않은 PR보다 **검토 소요를 줄이고, 개발자들이 자동 생성된 설명에 추가 편집을 더해 활용**하는 패턴이 확인되었다 [36]. 이는 문서화 부담을 AI가 덜어줌으로써 개발 생산성이 향상됨을 시사한다.

일상적인 **코드 리뷰 및 디버깅 작업**에도 AI 코드 설명이 활용되고 있다. 예컨대 Microsoft는 2023년 Visual Studio에 **"인텔리코드 AI 도움말"**을 도입하여, 예외 오류가 발생하면 해당 스택트레이스를 분석하고 **관련 코드의 원인을 자연어로 설명**해주거나, 복잡한 클래스의 용도를 요약하여 보여주는 기능을 시험했다 [37] [38]. 또한 **자동 문서화 도구**로서, Sphinx나 Javadoc 주석을 AI가 채워주는 시도도 나타나고 있다. Start-up인 **Mintlify** 등은 코드 리포지토리를 스캔하여 함수와 클래스의 역할을 추론하고 마크다운 문서를 생성하는 AI 서비스를 선보였고, Tabnine 등 기존 코드 완성 툴도 LLM을 탑재해 주석 보완 기능을 강화했다 [39].

개발자들의 반응은 대체로 긍정적이다. 2023년 GitHub 설문에 따르면 Copilot 사용자 중 상당수가 "코딩 작업 시간이 단축되고 코드 품질이 개선되었다"고 느꼈으며, MS리서치 실험에서는 **주어진 과제를 Copilot을 활용한 그룹이 그렇지 않은 그룹보다 평균 55% 빠르게 완료**하는 결과도 보고되었다(2022, Zimmerman 등). 이러한 효용 때문에 **대형 IT기업들은 사내 코드베이스용 AI 비서**를 구축하고 있다. 예를 들어 **Google**은 2023년 자사 코드 검색에 PaLM2 기반 AI 기능을 추가하여, 엔지니어가 수억 줄의 코드에서 특정 함수 사용 사례를 찾거나 모호한 코드를 물어보면 AI가 관련 맥락을 찾아 설명해주도록 했다. **아마존**도 CodeWhisperer를 통해 주석 생성과 보안 취약점 설명을 지원하고 있다.

결국 **코드 설명 생성 기술은 개발자의 일상 업무에 깊이 스며들고** 있다. 새 기능을 구현할 때 AI가 보일러플레이트 코드를 생성해주고, 개발자는 그 코드를 다듬는 데 집중한다. 작성한 코드에 **AI가 자동으로 주석과 문서화를 제안**하여, 개발자는 문서 작성 시간을 줄이고 로직 자체에 더 집중할 수 있다. 코드 리뷰 시에는 AI가 **변경 요약과 잠재적 문제를 지적**해주어 리뷰어의 부담을 덜고 있다. 이러한 **"사람+AI 협업"**은 개발 워크플로우 효율을 높일 뿐 아니라, 주석이나 문서를 간과하여 생길 수 있는 지식 손실을 방지하고 **프로젝트 지식을 체계화**하는 데 기여한다. 앞으로도 더 세밀한 코드 컨텍스트 이해와 자연어 서술 능력을 갖춘 모델이 등장하면서, 개발자는 **자신만의 AI 코드 비서에게 코드 이해를 맡기고 핵심 로직 설계에 전념**하는 방향으로 역할이 진화할 것으로 기대된다.

3. 사용자 ID 기반 접근 제어 기술 (RBAC vs ABAC)

RBAC와 ABAC의 개념 및 차이: 접근 제어 모델로 흔히 사용되는 **RBAC(Role-Based Access Control)**와 **ABAC(Attribute-Based Access Control)**는 **권한 부여 방식에서 근본적 차이**가 있다. **RBAC**는 말 그대로 **"역할" 기반으로 접근을 제어**한다. 즉 사용자가 어떤 사전 정의된 역할(role)을 가지고 있느냐에 따라 그에 할당된 권한 세트를 부여받는다 [40]. 예를 들어, 회사 시스템에서 관리자, 일반직원, 게스트 같은 역할별로 읽기/쓰기 권한을 미리 정해두고, 사용자에게 해당 역할을 부여하는 식이다. 반면 **ABAC**는 **사용자나 자원(Resource)의 속성(attribute)에 따라 규칙을 정하고, 조건을 만족하면 접근 허가**하는 방식이다 [40]. 사용자 속성은 부서, 직급, 고용형태, 보안등급, 근무지 등 다양하게 정의될 수 있고, 자원 속성은 문서의 기밀등급, 데이터의 생성 날짜, 요청 행위(읽기/쓰기), 접속 환경(시간, IP) 등까지 포함한다. **요약하면, RBAC는 "누가 어떤 역할인가"를 본다면, ABAC는 "누가 어떤 특성을 지녔고, 대상 자원이 어떤 특성을 가졌는가"를 모두 고려하여 세부 정책으로 접근 여부를 결정**한다 [40].

사내 시스템 적용 시 장단점: RBAC와 ABAC는 각각 장점과 한계를 지닌다. **RBAC의 최대 장점은 단순성과 성능**이다. 권한 규칙이 역할 단위로 단순화되어 있으므로 **시스템이 접근여부를 매우 빠르고 쉽게 판단**할 수 있고, 구현도 비교적 간단하다 [41]. 특히 조직 규모가 작거나 권한 구조가 직관적인 경우 RBAC만으로 충분하며, 실제로 많은 기업이 Active Directory 그룹 등의 형태로 RBAC를 기본권한체계로 활용해왔다. 하지만 RBAC는 **세밀한 통제에 한계**가 있다. 조직이 커지고 예외적인 권한 조합이 늘어나면 "역할 폭발(role explosion)" 현상이 발생한다 [42]. 즉 사소한 차이를 반영하기 위해 역할을 계속 추가하다 보면 수백·수천개의 역할이 난립하게 되어 오히려 관리가 어려워진다 [42]. 예를 들어 팀장(읽기전용), 팀장(쓰기권한), 부팀장, 부서이동직원 등 역할이 기하급수적으로 늘어날 수 있다. 반면 **ABAC의 강점은 유연성과 세분화**에 있다. 접근 규칙에 거의 **무제한에 가까운 다양한 속성을 조합**할 수 있으므로, **"정책으로 구현할 수만 있다면" 실제 업무 시나리오를 정밀하게 반영**할 수 있다 [43]. 또한 RBAC로는 표현 어려운 **시간 제한, 위치 기반 제약, 조직 구조와 무관한 특별 권한** 등을 ABAC는 손쉽게 적용할 수 있다 (예: "이 문서는 평일 근무시간에만 재무부서 직원들이 열람 가능"). **데이터 자체에 기밀도 태그를 달고 사용자 신원에 보안 등급을 부여한 뒤 일치 여부를 확인**하는 식으로 **데이터 중심 보안**을 구현할 수도 있다 [44]. 이러한 **높은 통제력** 때문에 미국 NIST 등의 보안 권고에서는 "적절히 구현된 ABAC가 정책 정확도를 높인다"고 평가한다 [45].

그러나 **ABAC의 단점은 복잡성과 초기 구축 비용**이다. 세부 규칙을 설계하려면 **다수의 속성 정의와 정책 수립에 전문성이 필요**하며, 시스템 도입 시 많은 시간과 노력이 들 수 있다 [46]. 속성 간 조합 규칙을 잘못 설정하면 오남용이나 권한 누락이 발생할 위험도 있어, **초기 설정을 신중히 하지 않으면 나중에 바로잡는 데 큰 비용이 든다** [46]. 예컨대 잘못된 정책으로 특정 조건에서 접근이 불필요하게 차단되면 업무 중단이 발생할 수 있다. 이런 이유로 많은 기업이 **기존 RBAC 체계를 완전히 버리지 않고 ABAC 요소를 부분 도입**하여 혼합 활용한다. Okta 등 IAM 업계도 "현실적으로 다수 기업은 상위 권한은 RBAC로 부여하고, 세부 제한은 ABAC로 거는 하이브리드 모델"을 권장한다 [47]. 실제 사례로, 미국 정부도 전통적으로 RBAC(그룹 기반) 인증을 사용하면서, Zero Trust 이행을 위해 ABAC 개념을 추가 적용하고 있다 [48] [49]. **ABAC를 전면 도입한 기관**으로는 미국 국방부(DOD)의 NSA, CIA 등이 있으며, 이들은 문서 분류와 사용자 인가등급을 매칭하는 XACML 기반 정책을 활용한다.

최근 보안 권장 사항: 2023년 발표된 미국 국가 사이버보안 전략(National Cybersecurity Strategy)과 제로트러스트 가이드라인 등에서는 **"정체성에 기반한 세분화된 접근 통제"**를 강력히 요구하고 있다 [50] [44]. 이는 기존의 "내부자=신뢰, 외부자=불신" 경계보안에서 탈피하여, **모든 접근을 맥락에 따라 검증**하라는 것이다. 이러한 Zero Trust 철학 아래에서는 **단순 RBAC보다는 ABAC (또는 정책 기반 접근통제, PBAC)**가 핵심 수단으로 부각된다 [44]. Virtru 등의 보안기업은 "제로트러스트 시대에 데이터 자체에 레이블을 붙여 보호하는 ABAC가 답"이라고 강조하며, **데이터 중심 접근통제**를 통한 침해피해 최소화를 제안하고 있다 [51]. 미 연방 CIO 협의회 역시 2023년 Zero Trust 지침에서 "ABAC 모델로의 전환이 필요"함을 명시하고, 사용자 위치, 시간, 기기 속성 등을 정책에 활용하도록 권고했다 [52]. 요약하면 최근 보안 트렌드는 **정적인 RBAC에서 동적인 ABAC (또는 하이브리드)**로의 이행이며, 이를 통해 권한남용을 줄이고 최소권한의 원칙을 구현하라는 것이다.

사내 PDF 저장소/RAG 검색엔진과 접근제어 통합: 내부 문서 검색용 RAG 시스템에 RBAC/ABAC를 적용하려면, **사용자 신원 정보와 문서 메타데이터를 연계**해야 한다. 우선 **RBAC 방식**으로 접근제어를 통합하는 방법은, **문서 색인 시 각 문서에 허용된 역할 정보를 메타데이터로 저장**하고, 사용자가 검색 질의할 때 그의 역할에 맞는 문서만 필터링하여 검색하는 것이다. 예를 들어 "기밀" PDF에는 metadata로 `"access_role": ["관리자","보안담당"]`를 달아두고,

검색시 로그인 사용자의 역할과 대조하여 일치 문서만 답변 생성을 위한 후보로 삼는다. 이러한 **메타데이터 기반 필터링은 Amazon Kendra 같은 엔터프라이즈 검색엔진이나 LlamaIndex, Pinecone 등에서 기본적으로 지원**된다 [53] [54]. 한 AWS 예시에서는, Slack 챗봇의 RAG 파이프라인에 Amazon Kendra(문서 검색)를 사용하고 **Kendra에 사용자 그룹 정보를 전달하여 필터링**함으로써 "챗봇이 사용자 권한이 있는 지식만 활용하도록" 구성했다 [55] [56]. 이는 **기존 IAM 시스템과 RAG를 연동**한 사례로, Thorn이라는 비영리단체가 AWS 솔루션을 활용해 구현하였다 [57] [56]. 결과적으로 직원들은 Slack에서 질문하면 AI가 즉시 답변하되, **본인이 볼 수 없는 자료는 검색 자체가 안 되도록** 되어 보안이 유지되었다.

ABAC 방식의 통합은 한층 정교하다. **문서마다 속성 태그를 부여**하고 **사용자 프로필에도 속성을 정의**한 뒤, 미리 정한 **정책 룰에 따라 매칭**하는 것이다. 예를 들어 사내 PDF 저장소에 모든 문서에 `"부서": "법무", "문서등급": "3"` 같은 태그를 붙이고, 사용자 계정에도 `"소속부서": "법무", "인가등급": "2"` 등의 속성을 갖게 할 수 있다. 정책엔진은 "`문서.부서 == 사용자.소속부서` 그리고 `문서.문서등급 <= 사용자.인가등급`"과 같은 규칙을 평가하여, 조건을 만족하는 문서만 LLM이 접근하도록 허용한다. 이러한 **속성기반 필터링**은 응용계층에서 구현하거나, 지원되는 벡터DB의 쿼리 필터로 실행할 수 있다. Databricks 벡터검색의 사례에서는 **질의 시 사용자 속성으로 메타데이터 필터를 구성해 전달하면, 검색 API 단계에서부터 사용자가 허가된 데이터만 반환**되도록 하고 있다 [58] [59]. "사용자와 역할을 벡터와 메타데이터로 연결"하는 방법도 있으나, Pinecone 등의 권고에 따르면 검색엔진에 보안 로직을 지니게 하기보다는 **검색 후 애플리케이션 레벨에서 걸러내는** 것이 권장된다 [60]. 이는 분리된 권한관리 서비스(예: OAuth/OIDC, OPA 등)가 **검색 결과를 후처리**하여 민감 정보는 응답 생성 전 제거하는 형태다 [61] [62].

또한 **RBAC와 ABAC 혼합** 적용도 가능하다. 예컨데 기본적으로는 **RBAC로 1차 거르고 ABAC 조건으로 2차 거르는** 방식이다 [63]. Virtru 블로그에서는 "일부 전통적 역할/그룹은 유지하면서, 문서에 태그를 달아 세분 권한을 적용하는 모델"을 사례로 들었는데, **기존 그룹권한으로 1차 제한을 둔 뒤, 문서 태그(예: 기밀도)에 따라 추가 속성검사를 하는 방식**이다 [63]. 이렇게 하면 오래된 시스템과의 호환성을 유지하면서도 필요한 곳에 속성 정책을 덧붙일 수 있다.

사례를 통한 이해: 가상의 예로, 사내 RAG 검색봇에 "재무 보고서 내용을 알려줘"라고 직원A가 물어본다고 하자. RBAC 통합의 경우, 봇은 먼저 A의 역할(예: 마케팅팀 직원)을 확인하고, 재무 보고서 문서들의 메타데이터에 지정된 허용 역할(예: 경영진, 재무팀)과 비교하여 A가 권한 없으면 **검색 단계에서 해당 문서를 제외**한다. ABAC 통합이라면, 재무 보고서에 태그된 속성(예: 부서=재무, 기밀등급=높음)과 A의 속성(부서=마케팅, 인가등급=중간)을 평가하여 **일치하지 않으므로 접근 거부**한다. 따라서 **LLM은 애초에 허가된 콘텐츠만을 가지고 답변 생성**을 시도하게 된다. 이렇게 하면 챗봇이 **부적절한 내부정보를 누설할 위험을 원천 차단**할 수 있다 [28].

구현 도구: 실제 구현에는 IAM 및 정책엔진을 활용한다. Azure, AWS, GCP 등은 각자 **문서 저장소와 IAM 연동 기능**을 제공하며, 예를 들어 AWS S3 + Kendra 조합에서는 S3 객체 ACL이나 AWS IAM 정책을 연계하여 Kendra 검색결과를 필터링할 수 있다. 오픈소스 솔루션으로는 **OPA(Open Policy Agent)**가 각광받는데, OPA의 정책언어(Rego)로 ABAC 규칙을 작성해두고 RAG 파이프라인의 검색 단계 후 결과들에 대해 `allowed = false`인 항목은 제거하는 식이다. Lucene 기반 검색엔진(Elastic/OpenSearch)도 **문서 색인 시 메타필드로 권한태그를 저장하고, 검색 쿼리에 사용자 태그 조건을 추가**하는 방식을 지원한다. Haystack 프레임워크는 쿼리 객체에 필터를 지정할 수 있어, 개발자가 `filter={"department": user.department}` 식으로 ABAC 필터를 걸어 답변을 생성하도록 할 수 있다.

정리: 사용자 ID 기반 접근제어는 사내 AI 시스템에서도 반드시 적용되어야 한다. RBAC와 ABAC 각각 장단점이 있으므로, 조직 규모와 보안 요구에 맞게 선택하거나 혼용한다. 최근 추세는 **정교한 ABAC로의 이행**이지만, 구현상의 복잡성을 고려해 **RBAC의 단순함과 ABAC의 유연함을 조화**시키는 전략이 현실적이다 [47]. 중요한 것은, 어떤 접근제어 모델이든 **사내 검색 AI의 응답 생성을 접근권한 틀 내로 한정**함으로써, **"알면 안 되는 사람에게까지 AI가 알려주는" 사태를 예방**하는 것이다 [28]. 실제 Thorn 사례나 Databricks 조언처럼, **기존 IAM/보안시스템과 AI를 통합**하면 기업은 AI 도입으로 인한 편의는 누리면서도 정보유출 리스크는 최소화할 수 있다 [56] [64]. Zero Trust 시대의 내부 AI 도구라면, 처음부터 보안을 설계에 포함시키는 것이 필수적이다. 이처럼 RBAC/ABAC 기반 접근제어와 RAG·AI 기술의 결합은, **기업이 안전하게 생성형 AI를 활용하는 기반**이 될 것으로 기대된다.

참고 문헌: Notion AI Q&A 사례 (Ivan Zhao 인터뷰, 2024) [2] [4]; Slack GPT 발표 (Salesforce, 2023) [9]; SAP Joule 보도 (SAP, 2024) [10]; Vectara 블로그 (Eva Nahari, 2024) [1]; AWS 내부 챗봇 사례 (George et al., AWS

Public Sector Blog, 2024) [55] [56] ; CodeT5+ 논문 (Wang et al., Salesforce, 2023) [30] ; StarCoder 논문 (Li et al., BigCode, 2023) [32] ; Phind 모델 블로그 (Royzen, Phind, 2023) [33] ; OpenCodeInterpreter 프로젝트 (Yue et al., 2024) [35] ; GitHub Copilot PR 논문 (Xiao et al., FSE 2024) [36] ; Okta 권한모델 해설 [65] [41] ; Virtru 제로트러스트 제언 (Zimmerman, 2023) [44] ; Okta 하이브리드 권장사항 [47] ; Pinecone 기술 블로그 (Schwaber-Cohen, 2024) [28] .

---

[1] [22] [26] [27] Enterprise RAG Predictions for 2025
https://www.vectara.com/blog/top-enterprise-rag-predictions

[2] [3] [4] [5] [6] Notion CEO: RAG is the future of knowledge management, AI brings SaaS into a new round of bundling - Eureka Blog
https://eureka.patsnap.com/blog/notion-ceo-rag-is-the-future-of-knowledge-management-ai-brings-saas-into-a-new-round-of-bundling-2/

[7] [8] June 18, 2024 – Notion 2.41: Notion AI now with Slack, GPT-4, and more!
https://www.notion.com/releases/2024-06-18

[9] Salesforce Announces Slack GPT, Unlocks Power of Conversational AI for Work - Salesforce
https://www.salesforce.com/news/press-releases/2023/05/04/slack-gpt-news/

[10] SAP Sapphire News Guide 2024 | SAP Business AI
https://www.sap.com/events/sapphire/innovation-guide/ai.html

[11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] LlamaIndex vs LangChain vs Haystack | by Hey Amit | Medium
https://medium.com/@heyamit10/llamaindex-vs-langchain-vs-haystack-4fa8b15138fd

[23] Enterprise privacy at OpenAI
https://openai.com/enterprise-privacy/

[24] [25] Introducing 100K Context Windows \ Anthropic
https://www.anthropic.com/news/100k-context-windows

[28] [54] [60] [61] [62] RAG with Access Control | Pinecone
https://www.pinecone.io/learn/rag-access-control/

[29] Introducing Code Llama, an AI Tool for Coding
https://about.fb.com/news/2023/08/code-llama-ai-for-coding/

[30] [2305.07922] CodeT5+: Open Code Large Language Models for Code Understanding and Generation
https://arxiv.org/abs/2305.07922

[31] [32] [2305.06161] StarCoder: may the source be with you!
https://arxiv.org/abs/2305.06161

[33] [34] Phind
https://www.phind.com/blog/phind-model-beats-gpt4-fast

[35] OpenCodeInterpreter
https://opencodeinterpreter.github.io/

[36] [38] Generative AI for Pull Request Descriptions: Adoption, Impact, and Developer Interventions
https://arxiv.org/html/2402.08967v1

[37] GitHub Copilot in Visual Studio: A Recap of 2023
https://devblogs.microsoft.com/visualstudio/github-copilot-in-visual-studio-a-recap-of-2023/

39 The Top LLMs For Code Generation: 2024 Edition
https://www.scribbledata.io/blog/the-top-llms-for-code-generation-2024-edition/

40 41 42 43 45 46 47 65 RBAC vs. ABAC: Definitions & When to Use | Okta
https://www.okta.com/identity-101/role-based-access-control-vs-attribute-based-access-control/

44 48 49 50 51 63 In A National Call for Zero Trust, ABAC is the Way Forward
https://www.virtru.com/blog/zero-trust/national-cybersecurity-strategy-attribute-based-access-control

52 [PDF] Advancing Zero Trust Maturity Throughout the User Pillar
https://media.defense.gov/2023/Mar/14/2003178390/-1/-1/0/CSI_Zero_Trust_User_Pillar_v1.1.PDF

53 Access control for vector stores using metadata filtering with Amazon ...
https://aws.amazon.com/blogs/machine-learning/access-control-for-vector-stores-using-metadata-filtering-with-knowledge-bases-for-amazon-bedrock/

55 56 57 Elevating internal customer support at Thorn with AWS: A generative AI use case | AWS Public Sector Blog
https://aws.amazon.com/blogs/publicsector/elevating-internal-customer-support-at-thorn-with-aws-a-generative-ai-use-case/

58 64 Mastering RAG Chatbot Security: ACL and Metadata Filtering with ...
https://community.databricks.com/t5/technical-blog/mastering-rag-chatbot-security-acl-and-metadata-filtering-with/ba-p/101946

59 Riding the RAG Security Trail: Access, Permissions & Context
https://www.lasso.security/blog/riding-the-rag-trail-access-permissions-and-context